

Building RESTful APIs with Spring MVC

Student Workbook 9a – REST and Spring Boot

Version 7.1 Y

Table of Contents

Module 1	Creating New Spring Boot Apps	4
Section 1–1	Bootstrapping Spring Boot	5
	Creating a New Spring Boot App	6
	Getting Started with Spring Boot	7
	Opening the Spring Boot App in IntelliJ	8
	What do you get?	10
	<code>static void main</code>	11
	Exercises – Bootstrapping Northwind Traders	13
Section 1–2	Understanding Dependencies	14
	Dependencies Explained	15
	Loading Dependencies into IntelliJ	16
	Exercises – Adding Dependencies	18
Section 1–3	Demo and Exercise App	20
	Conference App	21
	Northwind Traders API	22
Module 2	Adding Features	23
Section 2–1	Setting up the Structure	24
	Organizing with Packages	25
Section 2–2	Creating the Model	27
	Model	28
	Getters and Setters	30
Section 2–3	Creating the Repository	31
	Repository	32
	Exercises – Data in Northwind Traders	35
Section 2–4	Adding the Configuration	36
	Configuring a Spring Boot application	37
	Adding properties	38
	Exercises – Configuring the DB for Northwind Traders	39
	Using Properties	40
Section 2–5	Custom Properties Files and Profile	41
	Custom Properties Files	42
	Using profiles	43
	Example: Using Profiles	45
Section 2–6	Creating the Service	47
	Service	48
	<code>SessionService</code> Methods	50
Section 2–7	Creating the Controller	51
	Controller	52
	The HTTP method annotations	53
	<code>SessionController</code> Endpoints	54
	<code>SessionController</code> Endpoints	55
	Full Request Flow	56
	Exercises – Service and Controller for the Northwind Traders API	57
Module 3	Network Communication	59
Section 3–1	Network Communication	60
	Single Computer	61
	What is DNS	62
	<code>localhost</code>	63
	DNS on a small network	64
	What about the Internet?	66
	Communicating over the Internet	67

Section 3–2	The Request Response Model	68
What is a Web Server		69
What's in a URL		70
The Request		71
Request and Response		73
Section 3–3	Representational State Transfer or REST	74
Creating a RESTful Application		75
GET		76
POST		78
PUT		79
DELETE		80
Understanding the Request		81
Module 4	Understanding Spring Magic	82
Section 4–1	Spring Beans and the Spring Container	83
Spring Beans		84
Spring Beans		85
@Configuration and @Bean		86
Component Scanning		87
Exercises – Breaking our Northwind Traders API		88
Section 4–2	Working with Dependency Injection	90
Dependency Injection		91
Three Styles of Dependency Injection		93
Dependency Injection in the conferenceapp		94
What if Spring finds two beans of the same type?		95
Module 5	Continued Learning	96
Section 5–1	Authentication	97
What are JWT tokens?		98
Protecting Endpoints		100
Section 5-5	Practice Repository	101
Practice Problems		102

Module 1

Creating New Spring Boot Apps

Section 1–1

Bootstrapping Spring Boot

Creating a New Spring Boot App

- So far, we've worked with an existing Spring Boot application
- There are multiple ways to create a Spring Boot application from scratch:
 - Add the dependencies in the `pom.xml` manually
 - Use a wizard tool from IntelliJ ultimate, Eclipse or a plugin in VSCode
 - Go to start.spring.io, generate your project, and download it as a zip
 - * You will want to unzip and open the unzipped folder in your IDE
- We're going to work with start.spring.io, for this course
- Some of the wizard tools depend on this web tool too

Getting Started with Spring Boot

- Use start.spring.io to create a Spring Boot project from Scratch
- Fill out the form like in the screenshot (the versions may differ)
- In the “Dependencies” section, search for and add the “Spring Web”, “Spring Data JPA” and “MySQL Driver” dependencies as shown in the screenshot
- Hit the “Generate” button, download the zip, and unpack it into a folder on your computer.
- The download, if you selected Maven as your build tool, will be ready to go and will just require an import into your Eclipse

The screenshot shows the Spring Initializr web form. The form is divided into several sections: Project, Language, Spring Boot, Project Metadata, Dependencies, and a bottom bar with buttons.

Project

- ☐ Gradle - Groovy
- ☐ Gradle - Kotlin
- ☒ Maven

Language

- ☒ Java
- ☐ Kotlin
- ☐ Groovy

Spring Boot

- ☐ 4.1.0 (SNAPSHOT)
- ☐ 4.1.0 (M2)
- ☐ 4.0.4 (SNAPSHOT)
- ☒ 4.0.3
- ☐ 3.5.12 (SNAPSHOT)
- ☐ 3.5.11

Project Metadata

Group: Artifact:

Name:

Description:

Package name:

Packaging: ☒ Jar ☐ War

Configuration: ☒ Properties ☐ YAML

Java: ☐ 25 ☐ 21 ☒ 17

Dependencies

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

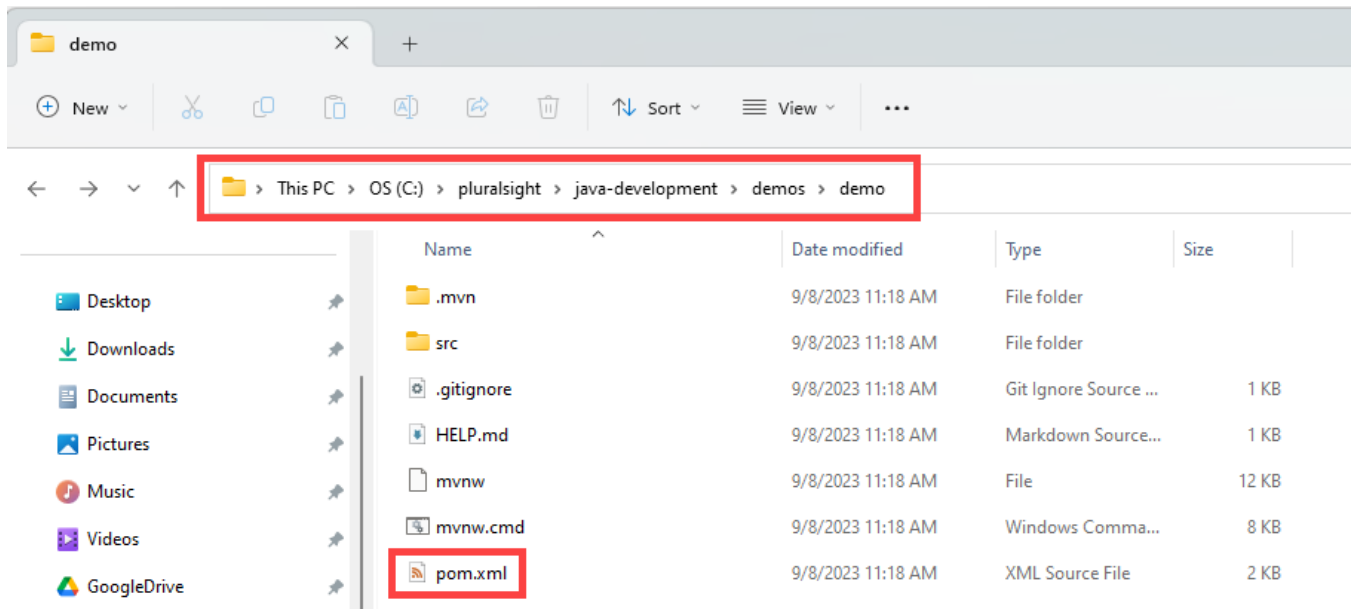
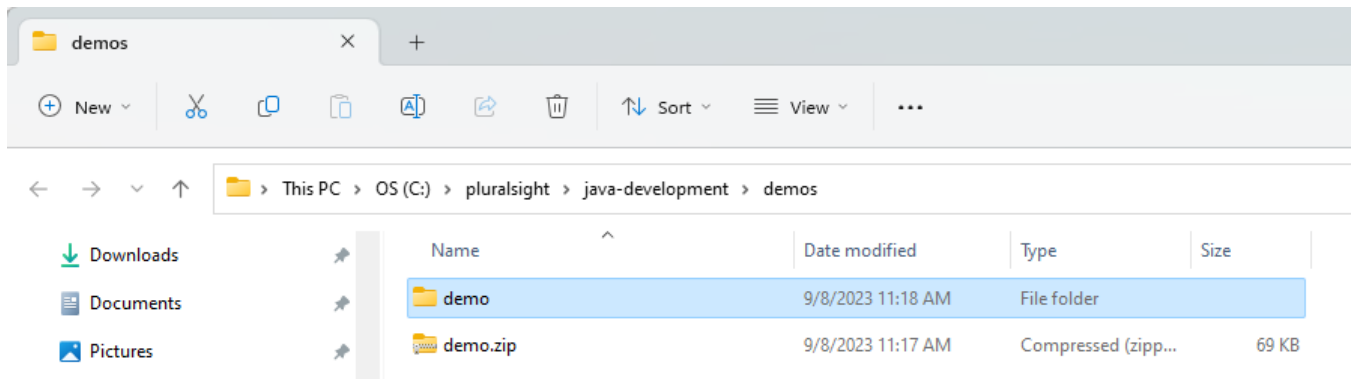
Spring Data JPA SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

MySQL Driver SQL
MySQL JDBC driver.

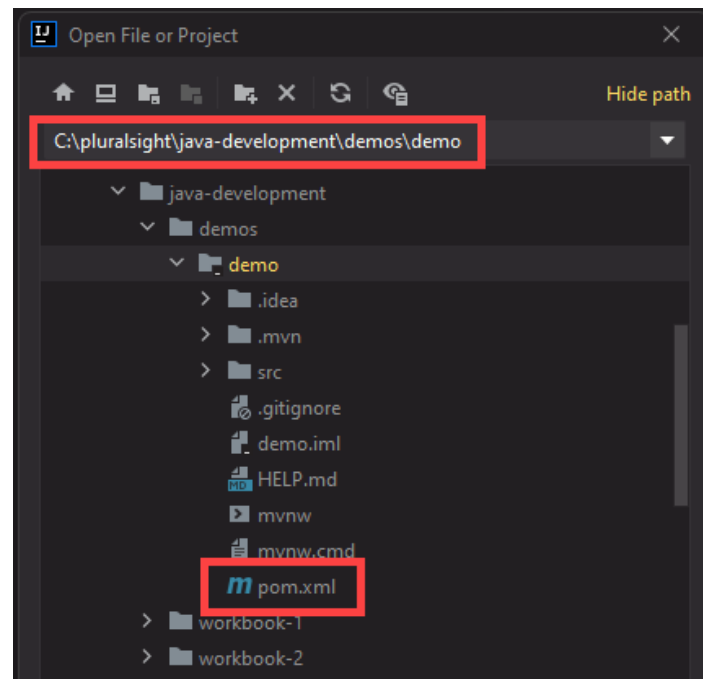
Buttons: GENERATE ⌘ + ↵, EXPLORE CTRL + SPACE, ...

Opening the Spring Boot App in IntelliJ

- Once downloaded, copy and unzip the file to your workspace
- Please note, the name of the zip in the screenshots is demo, yours will be `conferenceapp`, if you followed the instructions above

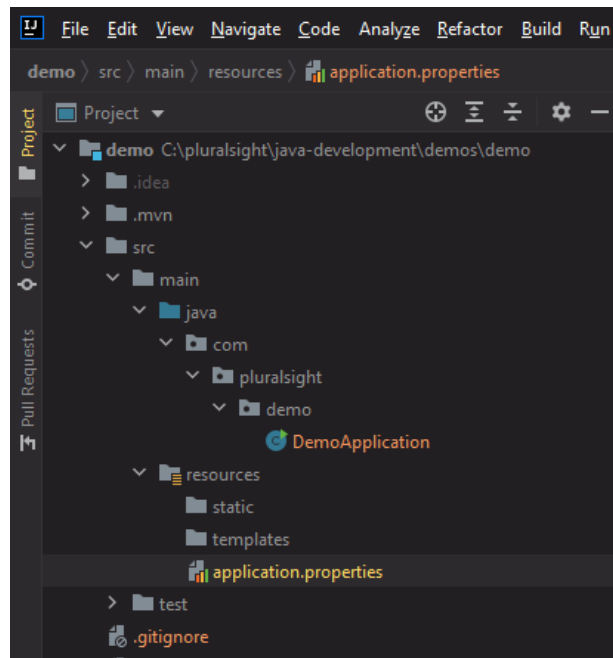


- In IntelliJ choose File -> Open and navigate to the project folder to open it
 - This is the folder that contains the `pom.xml` file



What do you get?

- A fully configured project with
 - All the initial settings made
 - All configurations set
 - And all of the libraries are included
- Included is also a *Maven binary*, which means you won't need to download and install Maven



static void main

- In a Spring Boot application the `static void main` method changes
- The only purpose of the `main` method is to launch the application

Example

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

- `SpringApplication.run` launches the application at its starting point
 - If no custom application is defined, Spring Boot launches a default application
- `SpringApplication.run` loads the `ApplicationContext`
- Spring Boot scans for any classes it needs to instantiate in the application and adds these to the `ApplicationContext`
 - These classes are called “beans”

Example

NOTE: to get access to the `ApplicationContext` you can just need to create a variable when you call the `run()` method. This is **not** something you'll do regularly in the `main` method in real-life use cases.

```
@SpringBootApplication
public class Application{

    public static void main(String[] args) {
        ApplicationContext context;
        context = SpringApplication.run(Application.class, args);
    }
}
```

Exercises – Bootstrapping Northwind Traders

Create a new folder in your `pluralsight` directory called `workbook-9`. Add code for the current week's exercises to this folder.

EXERCISE 1

You will create a new repository Java Maven project using Spring Boot. Open a browser and navigate to `https://start.spring.io`.

Configure your project with the following settings:

```
Selections:  Maven Project
              Java
              Latest stable Spring Boot version (no snapshot)
              Group: com.pluralsight
              Artifact: northwind-traders-springboot
              Jar
              Java Version 17
              Spring Web
```

Generate the project. Download and unzip it to your `workbook-9` folder.

Run the application to verify that your application runs.

Commit and push your code!

Section 1–2

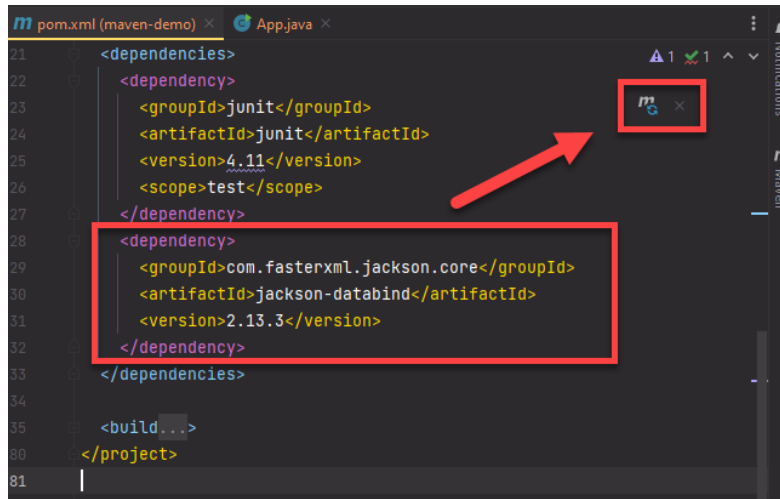
Understanding Dependencies

Dependencies Explained

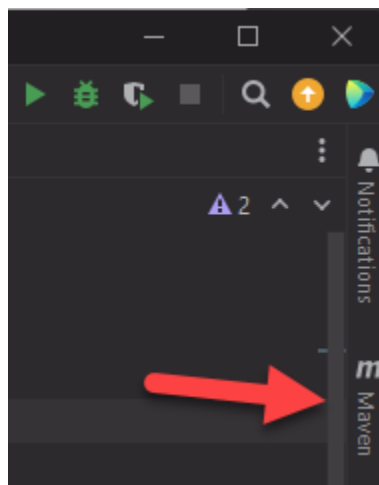
- A dependency is a library that adds specific functionality to your application
- Spring Boot uses a "starter" system that allows you to pick what you need, and it brings in everything required automatically
- In start.spring.io, you select starter dependencies
- Some dependencies we will use are:
 - **Spring Web**: Lets your application handle HTTP requests. Without it, no REST API
 - **Spring Data JPA**: Connects your Java objects to database tables. Write less SQL, work with objects instead
 - **MySQL Driver**: Connects your application to the MySQL database
- Without dependencies, Spring Boot is an empty shell
- Each dependency you add teaches your application a new skill. You pick the ones you think you will need during bootstrapping and you can always add more later
- Each dependencies you start with pulls in a set of related libraries with compatible versions. Spring Boot will manage version conflicts for you

Loading Dependencies into IntelliJ

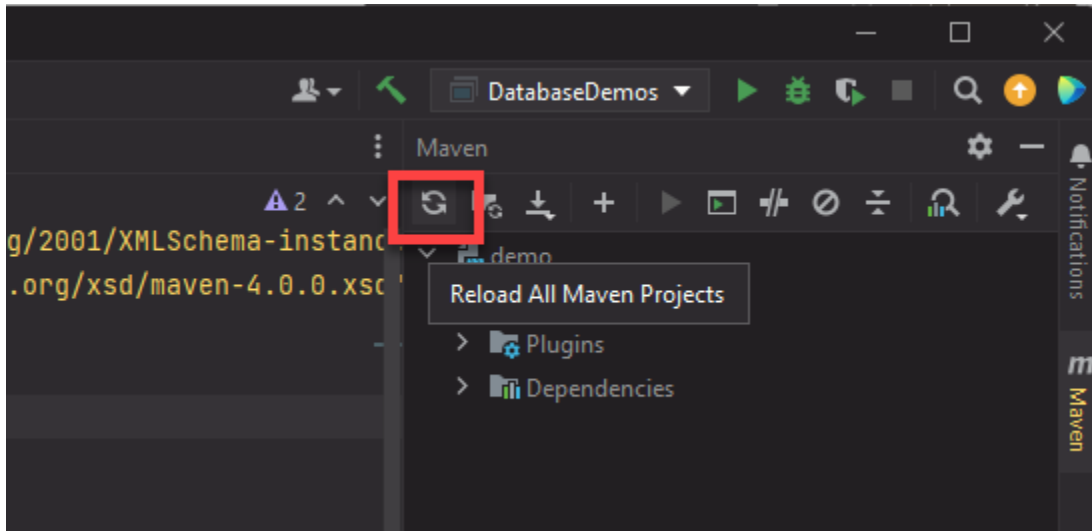
- This is very important! After you add a Dependency, like Jackson or Junit, you must tell IntelliJ / Maven to update
- You can click the floating Maven icon to do so



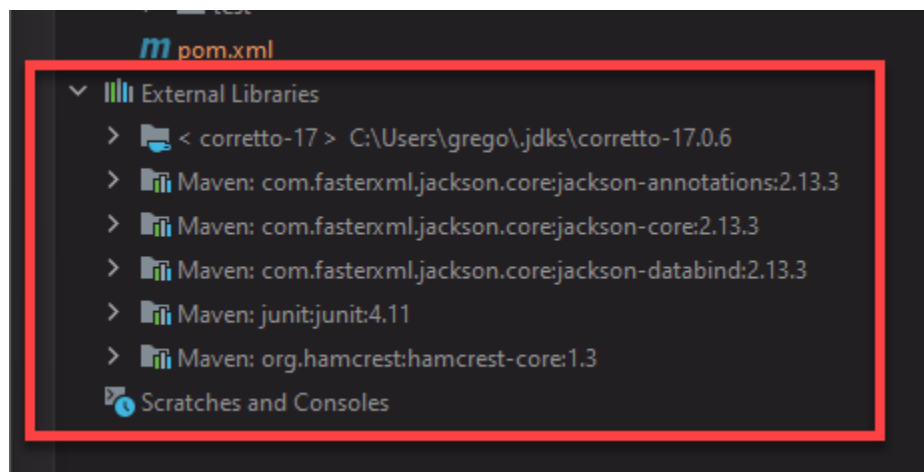
- Dependencies can also be reloaded from the Maven toolbox
 - Open the Maven toolbox



- Click the Reload All Maven Projects button



- You should be able to expand the External Libraries folder and view all the Spring jar files for your project



Exercises – Adding Dependencies

EXERCISE 1

You will add two new dependencies to your northwind-traders-springboot app

Go to your project in IntelliJ and open the pom.xml.

Make sure the dependencies list contains the following dependencies:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Make sure the dependencies are downloaded.

Run the application, you should get this error that will solve later:

```
*****  
APPLICATION FAILED TO START  
*****
```

Description:

```
Failed to configure a DataSource: 'url' attribute is not specified and no  
embedded datasource could be configured.
```

Commit and push your code!

Section 1–3

Demo and Exercise App

Conference App

- **Throughout the workbook, we'll be working with the example of a conference app**
- **The conference app is a RESTful API**
- **With this demo API we'll be able to:**
 - Get all sessions
 - Get all sessions by speaker or track
 - Get a session by id
 - Add a session
 - Update a session
 - Delete a session

Northwind Traders API

- For the exercises, you'll be creating a different project
- You've seen the Northwind Traders database
- You're going to create a RESTful API that will connect to this database

Module 2

Adding Features

Section 2–1

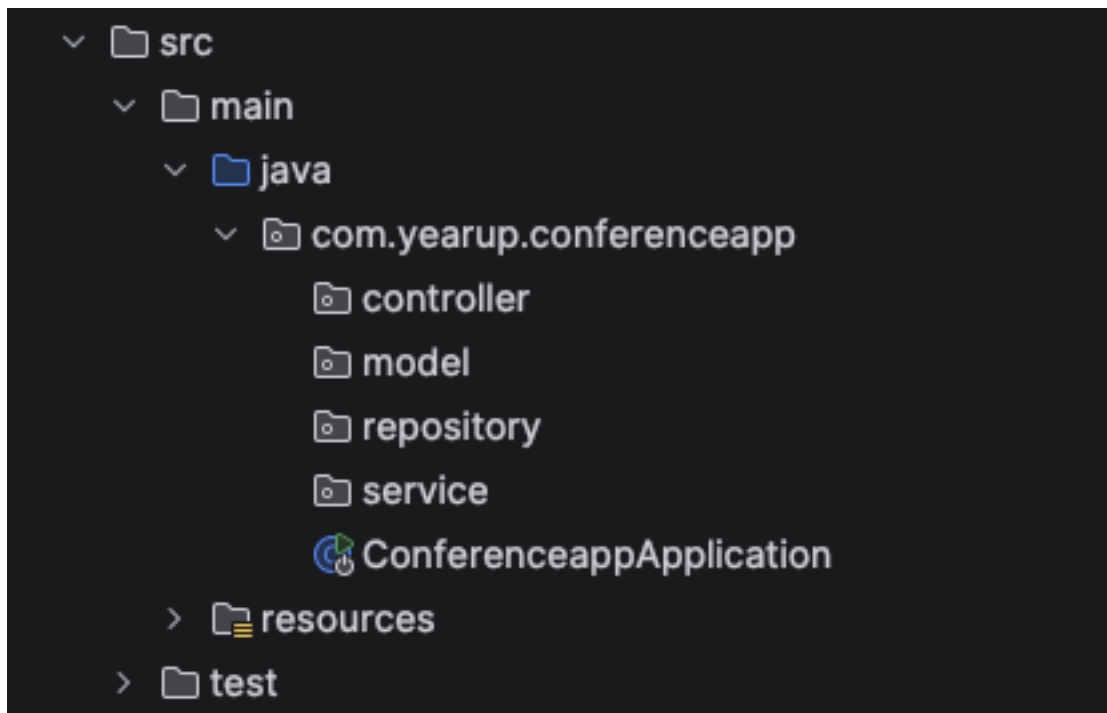
Setting up the Structure

Organizing with Packages

- **Packages organize our code into logical groups**
 - Without packages, all of your classes wind up in one folder. That might be manageable for 3 files, but it's chaotic with 30 files
- **Packages also control visibility**
 - Java's package-private default access can only work between classes in the same package
- **Two common approaches:**
 - **Package-by-layer:** Group by technical role such as controller, service, repository, model. You immediately know what a class does
 - **Package-by-feature:** Group by domain concept: session, speaker, registration. Each package contains its own controller, service, and repository. Better for larger applications as you immediately know what part of the app a class belongs to
- **Most tutorials use package-by-layer, and that's what we'll do too because it maps nicely to how Spring Boot works**
- **Our structure for the conference app:**

```
com.yearup.conferenceapp
├── controller    → receives HTTP requests
├── service       → contains business logic
├── repository    → talks to the database
└── model        → defines our data
```

- Create these four packages inside your main application package. They'll be empty for now and we'll fill them step by step



Section 2–2

Creating the Model

Model

- A model class represents your data
- In our example, the model defines what a "Session" looks like in our application. That includes what fields it has, what types they are. It's the blueprint for a session
- But... it's also a database table!
- With JPA, one class can serve double duty. The Java object is also the table definition. The annotations tell Spring how to map between the two.
- The annotations explained:
 - **@Entity**: Marks this class as a JPA entity. Spring now knows: this is something I need to manage and store.
 - **@Table(name = "sessions")**: Specifies the table name in the database. Without it, JPA uses the class name. Explicit is better.
 - **@Id**: This field is the primary key. Every entity needs one.
 - **@GeneratedValue(strategy = GenerationType.IDENTITY)**: The database generates the ID automatically. The database assigns the next available number on insert.
- The fields
 - No annotations needed on the other fields. JPA maps them automatically by name. For example, `private String title` becomes a `title` column. `private LocalDateTime`

`startTime` becomes `start_time`. JPA handles the naming conversion

Getters and Setters

- **JPA needs them to read and write field values. They might look like boilerplate and that's because they are**
 - For now, write them out. Later in your career you'll discover tools like Lombok or Java records that reduce this
 - We can create a `Session` class in our model package. Add the fields and annotations as shown.

```
package com.yearup.conferenceapp.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "sessions")
public class Session {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String speaker;
    private LocalDateTime startTime;
    private LocalDateTime endTime;
    private String room;
    private String track;

    // Getters and setters omitted
}
```

Section 2–3

Creating the Repository

Repository

- **A repository handles all database operations for an entity**
 - It's the layer that knows how to save, find, update, and delete your data.
 - The controller and service never write SQL, instead they ask the repository

- **The magic of Spring Data JPA**

- We write an interface, not a class
- No implementation needed. Spring generates it for us at runtime

```
public interface SessionRepository extends JpaRepository<SessionEntity, Long> {}
```

- `JpaRepository<SessionEntity, Long>`: Two items in the angle brackets are the entity type this repository manages, and the type of its primary key
- That's it. Spring now knows what table to talk with

- **By extending `JpaRepository`, we instantly have:**

- `findAll()` `findById()`, `save()`, `deleteById()`, `count()`, and more
- We'll write zero lines of code for all of that

- **We can create custom queries just by naming a method**

```
List<SessionEntity> findByTrackIgnoreCase(String track);
```

- If we read the method name, we can deduct what it actually does: "find by track, ignore case."
- Spring parses the method name and builds the query
- No SQL, no implementation. The method name is the query

- **Or another example:**

```
List<SessionEntity> findBySpeakerContainingIgnoreCase(String speaker);
```

- Containing adds a LIKE %value% search
- So this finds all sessions where the speaker name contains the search term, regardless of capitalization

- **The naming pattern**

- findBy + field name + optional keywords
- Some useful keywords:
 - * Containing
 - * IgnoreCase
 - * OrderBy
 - * Between
 - * GreaterThan

- **We can even chain the keywords together!**

```
List<SessionEntity> findByTrackAndRoom(String track, String room)
```

- If the method name compiles and follows the pattern, it works.
- If you misspell a field name, Spring fails at startup instead of silently at runtime

- **This is how to create a `SessionRepository` interface in our repository package:**

- It extends `JpaRepository`, and add the two custom query methods

```
package com.yearup.conferenceapp.repository;

import com.yearup.conferenceapp.model.Session;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface SessionRepository extends JpaRepository<Session, Long> {
    List<Session> findByTrackIgnoreCase(String track);
    List<Session> findBySpeakerContainingIgnoreCase(String speaker);
}
```

Exercises – Data in Northwind Traders

EXERCISE 1

You will be using the `northwind-traders-springboot` from the previous exercise.

Using the examples as a guide you will create admin capabilities for a store application to add products to your store. Create the following models.

`Product` - class - add the following members

`productId` - `int`

`name` - `String`

`category` - `String` (for now)

`price` - `double` (or `BigDecimal`)

`ProductRepository` - interface – make it extend the `JpaRepository`

If you run it, you'll still get the error. That's what we'll fix next, so don't worry about it.

BONUS (optional):

Add the ability to search for products by category, price, name, etc.

Commit and push your code!

Section 2–4

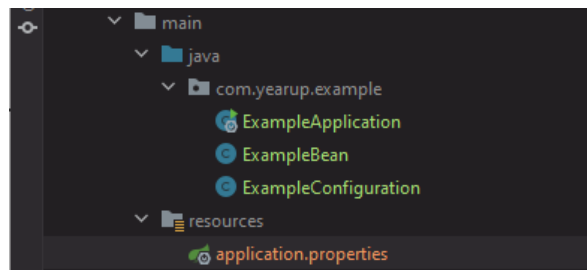
Adding the Configuration

Configuring a Spring Boot application

- **There are different ways to configure a Spring Boot application**
- **We are going to explore:**
 - Using properties
 - Using profiles
- **Please note, there are many ways to do this and we're only explaining a few here**

Adding properties

- We can use properties from a property file
- Property files are located in the `src/main/resources` dir
- Spring Boot includes a default file named `application.properties`



- The properties file is a configuration file that stores values you will need in your application
 - They are stored in `key=value` pair format
 - * `key=value`
 - * Each line holds a single key-value pair
 - It is common to store configuration variables that you do not want to have hard coded into your application - such as database connection

Example

```
spring.datasource.url=jdbc:mysql://localhost:3306/devdatabase
spring.datasource.username=localuser
spring.datasource.password=localpassword
```

Exercises – Configuring the DB for Northwind Traders

Continue working in your `northwind-traders-springboot` project.

EXERCISE 4

In this exercise you will add values to your `application.properties` file to fix the error and help us start the application.

First, open MySQL and create the schema:

```
CREATE SCHEMA sessions
```

Add key value pairs with the information needed to connect to your Northwind database. You will need the following key-value pairs:

```
spring.datasource.url=jdbc:mysql://localhost:3306/sessions
```

```
spring.datasource.username=<your-db-username>
```

```
spring.datasource.password=<your-db-password>
```

Run the application again, it should start successfully now with a message like this:

```
Started ConferenceappApplication in 1.509 seconds (process running for 1.766)
```

Commit and push your code!

Using Properties

- Spring properties are automatically picked up by Spring
- The properties can be used in classes by adding the source of the properties on top, using an annotation
- The property can be used with the `@Value` annotation

Example

```
// Note: we will not use this in our app

@Configuration
public class DatabaseConfig {

    private BasicDataSource basicDataSource;

    @Bean // We are configuring a DataSource Bean
    public DataSource dataSource(){
        return basicDataSource;
    }

    @Autowired // Each @Value is pulled in from the application.properties file
    public DatabaseConfig (@Value("${datasource.url}") String url,
                          @Value("${datasource.username}") String username,
                          @Value("${datasource.password}") String password)
    {
        basicDataSource = new BasicDataSource();
        basicDataSource.setUrl(url);
        basicDataSource.setUsername(username);
        basicDataSource.setPassword(password);

        // print the values to the screen just to verify it worked
        System.out.println(url + " : " + username + " : " + password);
    }
}
```

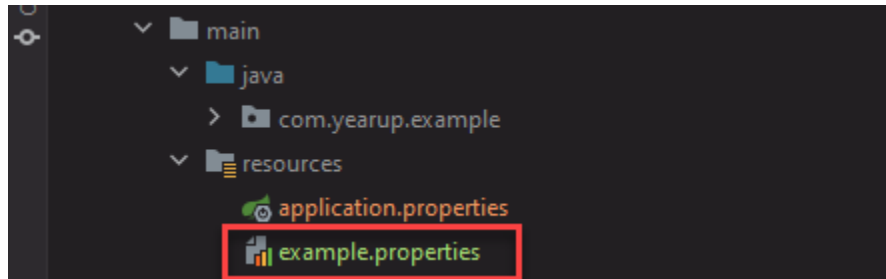
- In this example we configured a DataSource bean
 - * The connection string and user information was pulled from the `application.properties` file.

Section 2–5

Custom Properties Files and Profile

Custom Properties Files

- You can also create custom properties files
 - It must be a file with the extension `.properties`



- To read values from a custom properties files you must specify the file name with the `PropertySource` annotation
 - If no `PropertySource` is specified Spring Boot will search the `application.properties` file for keys
- This is a silly example, just for demonstrating how to use the custom properties file

Example

```
@Configuration
@PropertySource("classpath:example.properties")
public class ExampleConfiguration {

    @Bean
    public String valueProp(@Value("${greeting}") String hi) {
        return hi;
    }
}
```

Using profiles

- With properties we can configure our applications, but what if we wanted to have different configuration options available...?
- This is a common need, for example, different configurations for the development, test and production stages
- This can be done with profiles
 - We can map a bean to a certain profile
 - This is done with the `@Profile` annotation

Example

DevConfig.java

```
@Configuration
@Profile("dev")
public class DevConfig {
    public String setting = "dev";

    public DevConfig() {
        System.out.println("In DevConfig constructor: " + setting);
    }
}
```

TestConfig.java

```
@Configuration
@Profile("test")
public class TestConfig {
    public String setting = "test";

    public TestConfig() {
        System.out.println("In TestConfig constructor: " + setting);
    }
}
```

- There are many ways we can enable a certain profile, the one that we're going to be using is by adding a property to the `application.properties` file
- This property file is used for many configuration of a Spring Boot project

Example

```
spring.profiles.active=dev
```

- When we run the application we see the text from the `DevConfig` constructor in our output and not the text from the `TestConfig`

Example: Using Profiles

- **Given 3 different configuration files, you can quickly switch which database your application connects to**

– Just modify the `spring.profiles.active` value in the `application.properties` file

Example

database.dev.properties

```
connectionUrl=jdbc:mysql://localhost:3306/devdatabase
username=localuser
password =localpassword
```

database.test.properties

```
connectionUrl=jdbc:mysql://testserver:3306/testdatabase
username =testuser
password =testpassword
```

database.prod.properties

```
connectionUrl=jdbc:mysql://productionserver:3306/livedatabase
username =productionuser
password=$3cur3P@$w0rd
```

application.properties

```
spring.profiles.active=dev
```

- You need a different configuration class for each profile

DatabaseConfigDev.java

```
@Configuration
@Profile("dev")
@PropertySource("classpath:database.dev.properties")
public class DatabaseConfigDev {
    private BasicDataSource basicDataSource;

    @Bean
    public DataSource dataSource(){
        return basicDataSource;
    }

    @Autowired
    public DatabaseConfigDev (@Value("${connectionUrl}") String url,
                              @Value("${username}") String username,
                              @Value("${password}") String password)
    {
        basicDataSource = new BasicDataSource();
        basicDataSource.setUrl(url);
        basicDataSource.setUsername(username);
        basicDataSource.setPassword(password);
    }
}
```

DatabaseConfigTest.java

```
@Configuration
@Profile("test")
@PropertySource("classpath:database.test.properties")
public class DatabaseConfigTest {
    private BasicDataSource basicDataSource;

    @Bean
    public DataSource dataSource(){
        return basicDataSource;
    }

    @Autowired
    public DatabaseConfigTest(@Value("${connectionUrl}") String url,
                              @Value("${username}") String username,
                              @Value("${password}") String password)
    {
        basicDataSource = new BasicDataSource();
        basicDataSource.setUrl(url);
        basicDataSource.setUsername(username);
        basicDataSource.setPassword(password);
    }
}
```

Section 2–6

Creating the Service

Service

- **A service layer sits between the controller and the repository**
 - It contains our business logic
- **The business logic are the rules, validations, and decisions that make your application more than a pass-through to the database**
- **You might wonder, why not just call the repository from the controller?**
 - You could and it probably works. But the moment you need to add validation, send an email, combine data from two repositories, or handle an error gracefully... The controller becomes a mess. The service keeps that logic in one place
 - The controller should just be about handling the request
- **So, what belongs in a service?**
 - Mostly business and validation rules, including:
 - * Combining data from multiple repositories
 - * Transforming data before saving or returning it
 - * Any logic that isn't "receive request" (controller) or "talk to database" (repository)
 - For our example that could be:
 - * Is the session time slot available?
 - * Does the speaker exist?
 - * Is the starttime before the endtime?

- A service gets the repository injected through its constructor
- Spring handles the wiring and we only declare what we need

```
@Service
public class SessionService {

    private final SessionRepository sessionRepository;

    public SessionService(SessionRepository sessionRepository) {
        this.sessionRepository = sessionRepository;
    }
}
```

- Let's go over what's happening here
 - **@Service**: Tells Spring to manage this class as a bean
 - * Without it, Spring won't know it exists.
 - **Constructor Injection**: The recommended way to get dependencies.
 - * Spring sees the constructor, finds a matching bean, and passes it in
 - * No magic annotations on the field needed. It also makes our class easier to test and we can pass in a mock repository directly.

SessionService Methods

- Sessions need each operation to have its own method:

```
public List<Session> getAllSessions() {
    return sessionRepository.findAll();
}

public Optional<Session> getSessionById(Long id) {
    return sessionRepository.findById(id);
}

public Session createSession(Session session) {
    return sessionRepository.save(session);
}

public void deleteSession(Long id) {
    sessionRepository.deleteById(id);
}

public List<Session> getSessionsByTrack(String track) {
    return sessionRepository.findByTrackIgnoreCase(track);
}

public List<Session> getSessionsBySpeaker(String speaker) {
    return sessionRepository.findBySpeakerContainingIgnoreCase(speaker);
}
```

- Right now, most methods are thin wrappers around the repository
- That's fine for now and they give us a place to add logic later without touching our controller
- The value of the service layer grows as our application grows
- `Optional<SessionEntity> findById` might not find anything. Instead of returning `null`, Spring returns an `Optional`. It forces you to handle the "not found" case explicitly. We'll deal with that in the controller

Section 2–7

Creating the Controller

Controller

- **Think of the controller as a front door for your API**
- **It receives HTTP requests, calls the service, and returns a response**
- **If you find yourself writing business logic in a controller, that isn't a best practice**
 - Business logic belongs in the service
- **Spring knows it's a controller because of these annotations on top of the class:**
 - **@RestController**: This annotation does two things in one. First, this class handles HTTP requests, and second, every method returns data directly (as JSON), not a web page.
 - **@RequestMapping("/api/sessions")**: The base URL for all endpoints in this controller. Every method URL builds on top of this.
- **We need to inject the service in the controller**
- **Same pattern as the service with its repository. Constructor injection, Spring does the wiring**

```
private final SessionService sessionService;  
public SessionController(SessionService sessionService) {  
    this.sessionService = sessionService;  
}
```

The HTTP method annotations

- Each annotation maps to an HTTP verb. The verb tells the client what the operation means:
 - `@GetMapping`: Read data. Does not change anything
 - `@PostMapping`: Create something new
 - `@PutMapping`: Update existing data
 - `@DeleteMapping`: Remove something
- These verbs aren't Spring conventions, they're REST conventions
- A GET should never delete data
- A POST should never just return a list

SessionController Endpoints

- **Get all sessions:**

```
@GetMapping
public List<Session> getAllSessions() {
    return sessionService.getAllSessions();
}
```

- No path needed as it inherits `/api/sessions` from the class
 - * It returns the full list as JSON automatically

- **Get session by ID:**

```
@GetMapping("/{id}")
public ResponseEntity<Session> getSessionById(@PathVariable Long id) {
    return sessionService.getSessionById(id)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

- `@PathVariable` extracts `{id}` from the URL. A request to `/api/sessions/5` sets `id` to 5.
- `ResponseEntity` gives us control over the HTTP status code. Found the session? Return 200 OK. Didn't find it? Return 404 Not Found. This is where that Optional from the service pays off

- **Search by track:**

```
@GetMapping("/track/{track}")
public List<Session> getSessionsByTrack(@PathVariable String track) {
    return sessionService.getSessionsByTrack(track);
}
```

SessionController Endpoints

- **Search by speaker:**

```
@GetMapping("/speaker/{speaker}")
public List<Session> getSessionsBySpeaker(@PathVariable String speaker) {
    return sessionService.getSessionsBySpeaker(speaker);
}
```

- **Create a session:**

```
@PostMapping
public Session createSession(@RequestBody Session session) {
    return sessionService.createSession(session);
}
```

- `@RequestBody` takes the JSON from the request body and converts it into a `Session` object. The client sends JSON, your method receives a Java object. Spring handles the conversion

- **Delete a session:**

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteSession(@PathVariable Long id) {
    sessionService.deleteSession(id);
    return ResponseEntity.noContent().build();
}
```

- 204 No Content is the standard response for a successful delete. The resource is gone, there's nothing to return

Full Request Flow

- Client sends `GET /api/sessions/5`
- Spring routes to `getSessionById`
- Controller calls `sessionService.getSessionById(5)`
- Service calls `sessionRepository.findById(5)`
- Database returns the row (or not, if not found)
- Response travels back to the client as JSON
- Every layer does one job. That's the whole architecture

Exercises – Service and Controller for the Northwind Traders API

EXERCISE

You will continue working in your `northwind-traders-springboot` project. Using the demo as a guide, you will add a service and controller for your Product entity.

STEP 1: CREATE THE PRODUCTSERVICE

Create a `ProductService` class in your service package.

The service should:

- Inject the `ProductRepository` through the constructor
- Be annotated so Spring recognizes it as a bean
- Include methods for the following operations:
 - Get all products
 - Get a product by ID (think about what happens if the product doesn't exist)
 - Add a new product
 - Delete a product by ID

STEP 2: CREATE THE PRODUCTSCONTROLLER

Create a `ProductsController` class in your controller package.

The controller should:

- Be set up as a REST controller with a base path of `/api/products`
- Inject the `ProductService` through the constructor
- Include the following endpoints:

Verb	Path	What it does	Returns
GET	<code>/api/products</code>	Get all products	List of products
GET	<code>/api/products/{id}</code>	Get a product by ID	Product or 404
POST	<code>/api/products</code>	Add a new product	The saved product
DELETE	<code>/api/products/{id}</code>	Delete a product by ID	204 No Content

STEP 3: TEST YOUR API

Run your application. Using Insomnia (or your preferred API testing tool), test each endpoint:

- GET `http://localhost:8080/api/products` → should return an empty list or your existing products
- POST `http://localhost:8080/api/products` → send a JSON body to create a product
- GET `http://localhost:8080/api/products/1` → should return the product you just created
- DELETE `http://localhost:8080/api/products/1` → should return 204

Hint for the POST body:

```
{
  "name": "Chai Tea",
  "category": "Beverages",
  "price": 18.00
}
```

BONUS EXERCISE

If you added search methods to your repository in the previous exercise (like searching by category or by name), expose them through your service and controller now.

Think about: what verb do you use? What does the URL path look like? Look at how the conference app handles `/track/{track}` and `/speaker/{speaker}` for inspiration.

Commit and push your code!

Module 3

Network Communication

Section 3–1

Network Communication

Single Computer

- **Even applications that run on a single computer often need to communicate with each other**
 - Your Java Application with your MySQL Database
- **Think of your computer like an industrial complex which leases office space**
 - You can install many applications on your computer
 - Each application is like a business leasing office space
 - All applications have the same physical address (your computer)
 - But each "office" can have an internal address (like an apartment or unit number)
 - * The unit number is comparable to a port number on a computer
- **MySQL is a standalone application that allows other applications to interact with it**
 - The server address of MySQL installed on the local machine is `localhost`
 - MySQL listens on port `3306`

What is DNS

- **Each computer, like a business complex, has an ip address**
 - Ip address `127.0.0.1` refers to itself
 - Often referred to as Home
 - "There's no place like 127.0.0.1"
- **Your computer has a local DNS (Domain Name Server)**
 - This allows you to store aliases (or nicknames) for known ip addresses
 - `localhost` is an alias for `127.0.0.1`

Example

The following strings represent a JDBC connection string to a MySQL server that is installed on the same computer as the running computer.

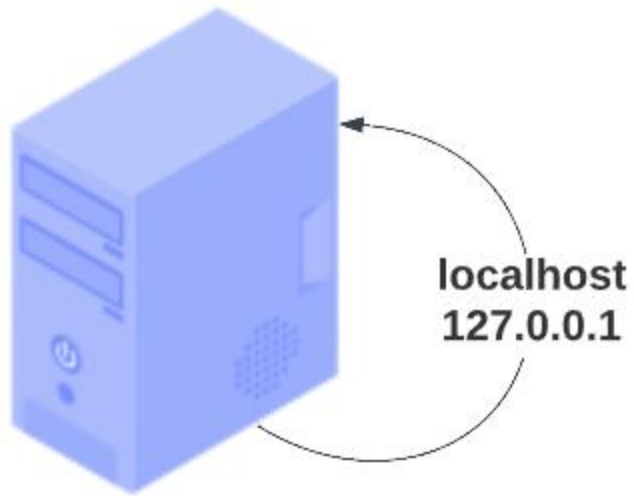
```
jdbc:mysql://localhost:3306/mydatabase  
  
equals  
  
jdbc:mysql://127.0.0.1:3306/mydatabase
```

The following urls represent a link to a website that is running on the same computer as the browser.

```
http://localhost/homepage.html  
  
equals  
  
http://127.0.0.1/homepage.html
```

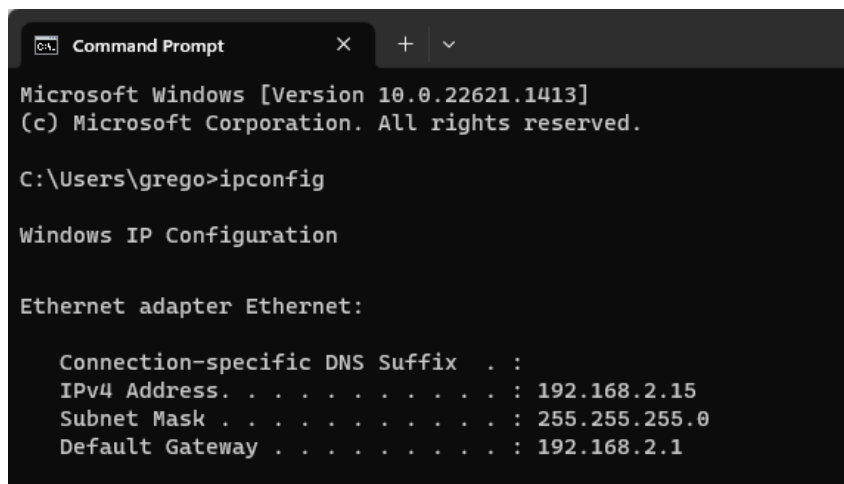
localhost

- **Localhost refers to the running computer**



DNS on a small network

- **When your computer is part of a small home or corporate network your computer is assigned an IP address**
 - This is so that computers on the same network can communicate with each other
- **The network has a Domain Name Server (DNS)**
 - The DNS assigns ip addresses to each computer on the network
 - Common addresses are 192.168.2.1 to 192.168.2.255
 - Run `ipconfig` to see what your network ip address is



```
Command Prompt
Microsoft Windows [Version 10.0.22621.1413]
(c) Microsoft Corporation. All rights reserved.

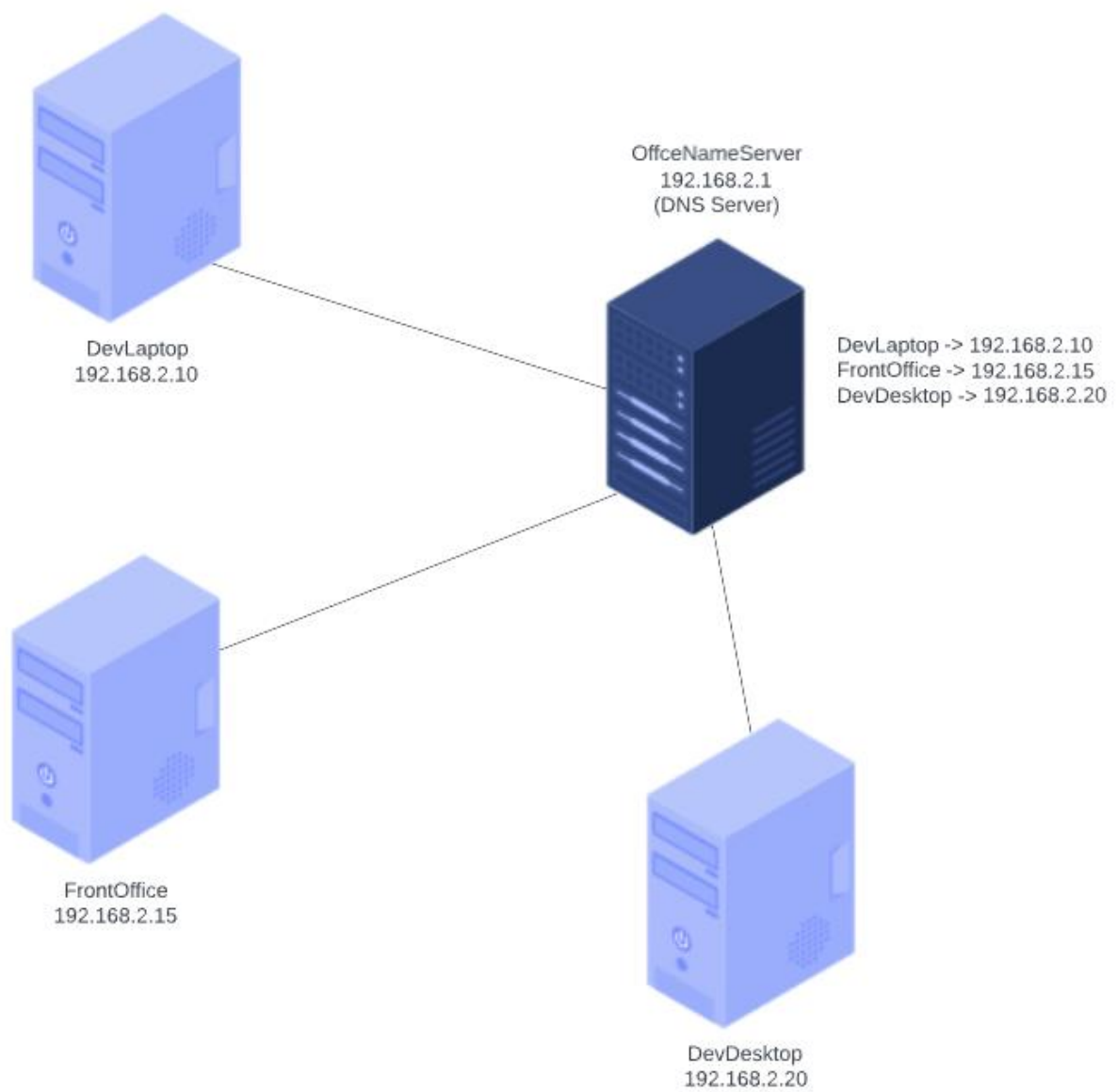
C:\Users\grego>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    IPv4 Address. . . . . : 192.168.2.15
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.2.1
```

- **The DNS can also assign aliases to each ip address on the network**
 - DevDbServer -> 192.168.2.15
 - TestDbServer -> 192.168.2.20



What about the Internet?

- **Each network that is connected to the internet also has an ip address that is unique**
 - Go to whatsmyip.com to discover your global IP address
- **If you know the IP address you can communicate with ANY computer that is connected to the internet**
- **Anyone can register a domain name to create a friendly, easy to remember name**
 - Use a registrar such as godaddy.com to purchase a domain name
 - Domain names are useful for websites, custom email addresses and more
- **When you register a domain name you can have it point to any ip address**
 - `my-custom-domain.com -> 75.286.xx.xxx`
 - There are several global domain name servers that keep a list of all registered domains and where they point
- **Internet Service Providers maintain their own DNS servers**
 - They keep a local copy (cache) of domains and ip addresses

Communicating over the Internet

- One computer can communicate with another computer on the internet

– The dev laptop can navigate to `www.my-custom-domain.com` if the domain is registered and pointing to the correct public ip address



Section 3–2

The Request Response Model

What is a Web Server

- **MySQL is a database server application that manages database access**
 - MySQL listens on port 3306 by default
 - The purpose of the database server is to give users access to data
- **A web server is an application that accepts incoming requests and responds to them**
 - The concept is similar to a database server
 - Web Servers generally listen either on port 80 or port 443
 - The purpose of web servers is to give users access to web applications
- **Web Servers can host web sites**
 - www.pluralsight.com
 - www.espn.com
 - etc.

What's in a URL

- The URL (Uniform Resource Locator) is made up of several parts that define the request



- **Protocol** - specifies how the data will be transmitted
- **Domain** - the domain name or ip address of the web server
- **Port** - the port number on which the server is listening
 - Port 80 is the default port for http
 - Port 443 is the default port for https
 - If a default port is in use, you do not need to add it to the url
 - * `http://www.my-custom-domain.com/path`
- **Path** - the defined path for a resource on the server
 - This often looks like a folder path to an html page
- **Query String** - additional query information
 - Key / value pairs of data passed to the server
 - These are often used like a filter in a WHERE clause of SQL

The Request

- **A web browser is an application that can request and view web pages**
 - The browser uses a URL to request a page
 - When the page is returned from the server the browser displays the result
- **The browser can make 2 types of requests**
 - A simple request to request information from the server
 - * `https://www.espn.com` - simple request
 - Send information to save or update on the server
 - * Registration information from a form

First Name:	
Last Name:	
Email:	
Address:	
City:	
State:	▼
Zip:	
Register	

- **Form data is sent with the request**

- **The full request is a basic text stream that is made up of 2 parts**

- header - contains the address of the resource you are requesting
 - * It also contains other bits of helper information that describes the request
- The body - this is optional information that is added at the end of the request
 - * It is a way to pass additional data or even attachments to the request
 - * If the request contains a body, it is added after a single empty line

Example

This is a simple request sent to the server. There is no body

```
GET https://www.my-custom-domain.com
```

Example

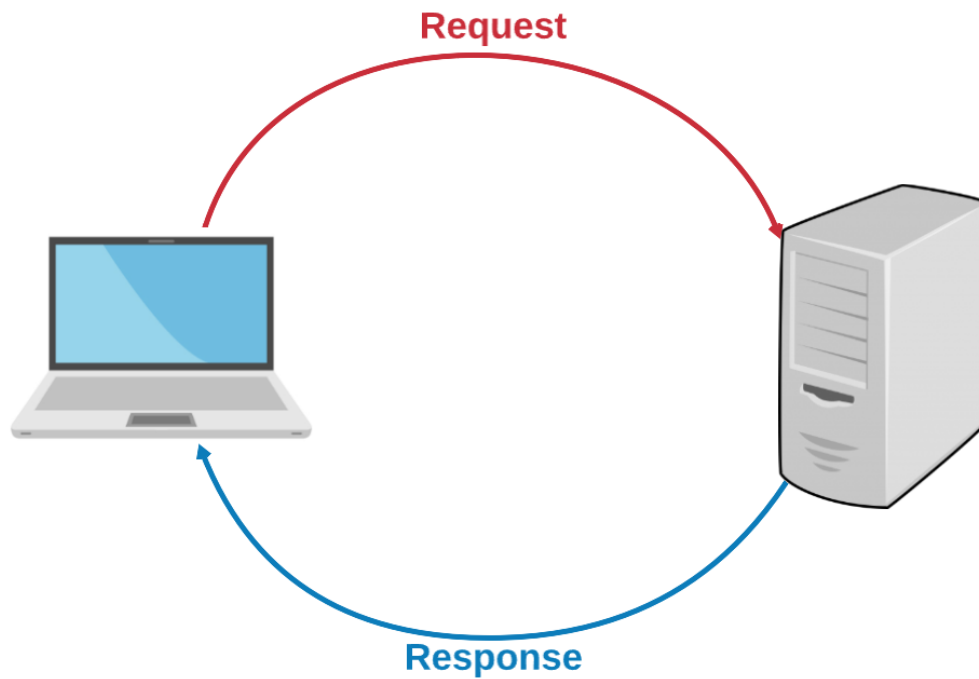
A request that sends form data in the body might look like this.

```
POST https://www.my-custom-domain.com/register
Content-Type: application/x-www-form-urlencoded

firstname=Carrie&lastname=Johnson&email=c.johnson@email.com
&address=123 Main Street&city=Salt Lake City&state=UT&zip=84111
```


Request and Response

- The request / response programming model requires that the client initiates a request from the server
 - To complete the request the client awaits a response
- Each request is a new cycle



Section 3–3

Representational State Transfer or REST

Creating a RESTful Application

- **Web applications generally follow the request / response model**
- **REST is an architecture style of programming web applications**
 - REST - Representational State Transfer
 - Set of guidelines that is widely adopted (though not an official web standard)
- **REST is used to expose CRUD operations for resources on a server**
 - The verbs are the actions that resemble CRUD operations in SQL statements
 - * GET = SELECT
 - * POST = INSERT
 - * PUT = UPDATE
 - * DELETE = DELETE
- **Using rest to access resources**
 - The URL defines the resource you want to access
 - * `https://www.my-custom-domain.com/products`
 - * specifies access to the **products** resource on the web server
 - * A resource often refers to a table in a database
 - The VERB defines what you want to do with the resource

GET

- When you GET a resource you are requesting one or more items
- A GET request never includes a body



- The response includes the requested data in a body



- **The URL defines which resource you want to access**

- `http://www.my-domain.com/products`

- * All products

- `http://www.my-domain.com/products?id=1`

- * A single product with id 1

- `http://www.my-domain.com/products/1`

- * A single product with id 1

- * Since finding a resource by id is extremely common this is standard alternative to using the query string

- `http://www.my-domain.com/products?category=beverages`

- * All products in the beverage category

- `http://www.my-domain.com/products?category=beverages&price_gte=10`

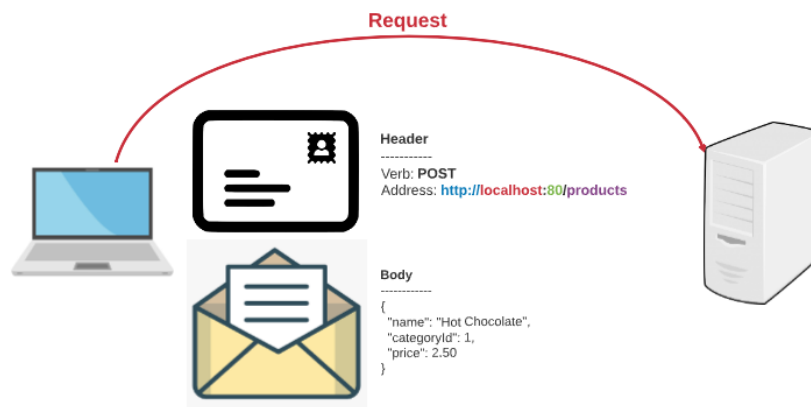
- * You can specify multiple query string criteria by separating the key=value pairs with **&**

- * All products in the beverage category

- * AND price is greater than or equal to \$10

POST

- When you POST a resource you are inserting a new item to that resource
- A POST includes a body of the data that you want to insert

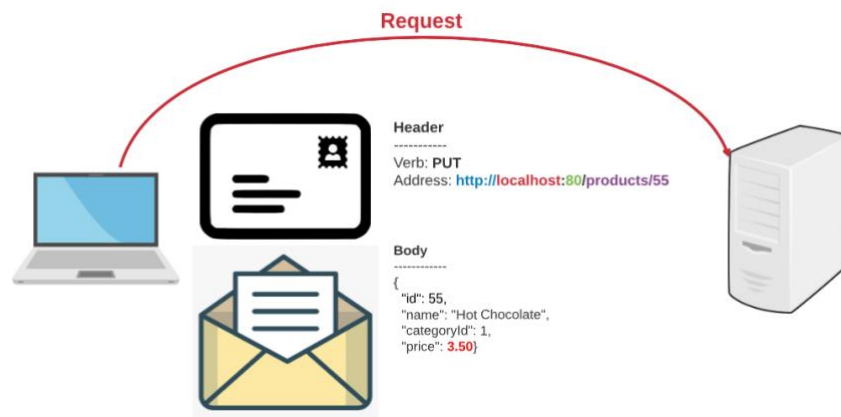


- The response includes the posted data in a body

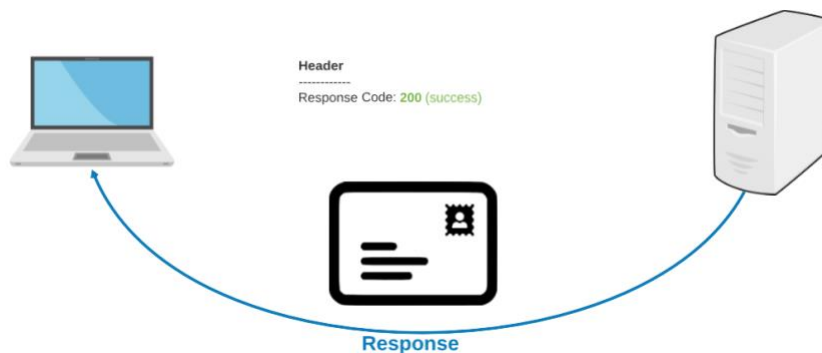


PUT

- When you PUT a resource you update an existing item in that resource collection
- A PUT includes the id of the resource you want to update, and a body of the data that you want to update



- The response DOES NOT include a body

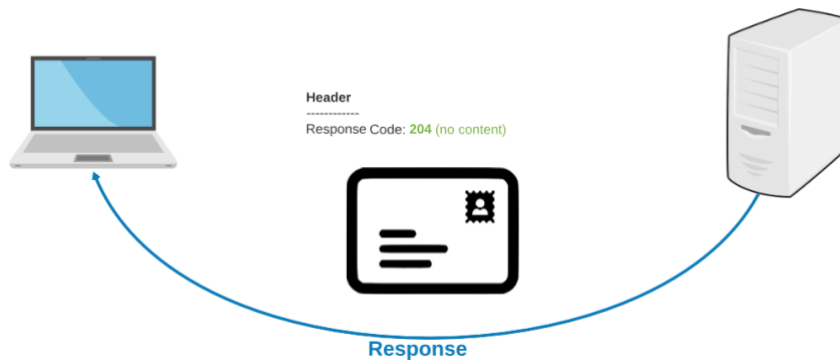


DELETE

- When you DELETE a resource you specify the id of the resource you wish to delete
- A DELETE does not include a body with the request



- The response DOES NOT include a body



Understanding the Request

- **The Verb and the URL in the request provide the critical information about what you want to accomplish**
- **Both the client and the server must agree on what the client expects and what the server provides**
- **Using REST provides standard practices so that developers have all of the information they need to program the server**

Module 4

Understanding Spring Magic

Section 4–1

Spring Beans and the Spring Container

Spring Beans

- Up until now, we've been adding annotations like `@Service`, `@RestController`, and `@Repository` without fully explaining what they do behind the scenes
- The two core concepts that make it all work are Spring Beans and Dependency Injection
- A Spring Bean is simply an object that Spring creates and manages for you. Instead of you writing `new SessionService()`, Spring does it
- Spring stores the service instance and hands it out wherever it's needed

Spring Beans

- **Beans live in the Application Context**
- **Think of it as a big storage box that Spring fills at startup**
- **It scans your project, finds all the classes it should manage, creates instances of them, and keeps them ready to use**
- **Spring know which classes to manage by annotations. You've already used several of them:**
 - `@Component` marks any class as a Spring Bean
 - `@Service` is a specialized `@Component`. Functionally identical, but signals "this class contains business logic." We used this on `SessionService`
 - `@RestController` is another specialized `@Component`. It signals "this class handles HTTP requests." We used this on `SessionController`
 - `@Repository` is yet another specialized `@Component`. Signals "this class talks to the database." We used this on `SessionRepository`
- **They are all components and that's why they all tell Spring to create and manage an instance of this class**

@Configuration and @Bean

- Sometimes you need more control over how a bean is created. In a `@Configuration` class, you can use `@Bean` on a method to manually define what Spring should store
- The return value becomes the bean
- The method name becomes the bean's name
- This approach is useful for third-party classes you can't annotate, or when you need to configure something specific before handing it to Spring
- To demonstrate the pattern:

```
@Configuration
public class AppConfig {
    @Bean
    public String appName() {
        return "Conference Planner";
    }
}
```

- Please note, it's not common to create beans of type `String` like this, a property in `application.properties` would make more sense. It's just a simple example for demonstrating the concept
- For your own classes, `@Service`, `@RestController`, and `@Repository` are usually the right choice

Component Scanning

- Component scanning is where Spring is looking for beans
- Spring scans for beans starting from the package that contains the `@SpringBootApplication` class
- Everything at that level or in sub-packages gets found. If a class lives outside that package tree, Spring won't see it
- This is why your package structure matters:

<code>com.pluralsight.conferenceplanner</code>	← <code>@SpringBootApplication</code> lives here
├── <code>controller</code>	← found
├── <code>service</code>	← found
├── <code>repository</code>	← found
└── <code>model</code>	← found

- If you accidentally place a class in `com.pluralsight.other`, Spring won't find it

Exercises – Breaking our Northwind Traders API

EXERCISE

In this exercise you will intentionally break your application to see component scanning in action.

STEP 1: MOVE THE MAIN CLASS

In your `northwind-traders-springboot` project, create a new subpackage called `startup` inside your main package.

For example: `com.pluralsight.northwind-traders-springboot.startup`

Move your `main` application class (the one with `@SpringBootApplication`) into this new package.

Your structure should now look something like this:

```
com.pluralsight.northwind-traders-springboot
├── controller
├── service
├── repository
├── model
├── startup
│   └── northwind-traders-springboot.java
```

STEP 2: RUN THE APPLICATION

Start the application. It should start without errors and Spring Boot itself still works fine.

STEP 3: TEST THE ENDPOINTS

Open Insomnia and try GET `http://localhost:8080/api/products`.

What happens? You should get a 404 Not Found or a Whitelabel Error Page instead of your product list

STEP 4: EXPLAIN WHY

Before you fix it, write down your answer to these questions (in comments in your code, on paper, or in a text file):

1. Why can't Spring find your controller, service, and repository anymore?
2. Where does Spring start scanning for beans by default?
3. Why did the application still start up without errors, even though none of your beans were loaded?

STEP 5: FIX IT

Move the main application class back to the root package
`com.pluralsight.northwind-traders-springboot`

Verify your endpoints work again.

Section 4–2

Working with Dependency Injection

Dependency Injection

- **Look at our `SessionController`. It needs a `SessionService` to do its job. And `SessionService` needs a `SessionRepository`**
- **Without Spring, you'd have to create all those objects yourself:**

```
SessionRepository repo = new SessionRepository(); // won't even work, interface
SessionService service = new SessionService(repo);
SessionController controller = new SessionController(service);
```

- **This is messy, especially when many classes depend on each other**
- **Dependency Injection means Spring handles this wiring for you**
- **When Spring creates the `SessionController` bean, it looks at the constructor:**

```
public SessionController(SessionService sessionService) {
    this.sessionService = sessionService;
}
```

- Spring sees: "this class needs a `SessionService`." It checks the Application Context, finds the `SessionService` bean it already created, and passes it in
- **That's dependency injection, Spring injects the dependency**

- The same thing happened one level deeper. When Spring created `SessionService`, it saw it needed a `SessionRepository` and injected that too
- You've been using dependency injection the whole time!
- The constructors we wrote in this workbook used dependency injection!
- Here, Spring injects `SessionRepository` into `SessionService`:

```
public SessionService(SessionRepository sessionRepository) {  
    this.sessionRepository = sessionRepository;  
}
```

- Spring injects `SessionService` into `SessionController`:

```
public SessionController(SessionService sessionService) {  
    this.sessionService = sessionService;  
}
```

- This is called constructor injection and it's the recommended approach. You declare what you need in the constructor, and Spring provides it

Three Styles of Dependency Injection

- **Constructor injection (recommended):**

```
@Service
public class SessionService {
    private final SessionRepository sessionRepository;

    public SessionService(SessionRepository sessionRepository) {
        this.sessionRepository = sessionRepository;
    }
}
```

- Fields can be final, which means they can't be changed after construction. Clear, safe, and easy to test

- **Field injection (not recommended for application code):**

```
@Service
public class SessionService {

    @Autowired
    private SessionRepository sessionRepository;
}
```

- Shorter, but the field can't be final, and you can't easily pass in a mock for testing. You'll see this in older tutorials, but avoid it in new code

- **Setter injection (rare):**

```
@Service
public class SessionService {
    private SessionRepository sessionRepository;

    @Autowired
    public void setSessionRepository(SessionRepository sessionRepository) {
        this.sessionRepository = sessionRepository;
    }
}
```

- * Only useful when a dependency is truly optional. In practice, you'll rarely need this

Dependency Injection in the conferenceapp

- Scans packages, finds `SessionRepository`, `SessionService`, `SessionController`
- Next Spring:
 - Creates `SessionRepository` (Spring Data generates the implementation)
 - Creates `SessionService`, injects the repository
 - Creates `SessionController`, injects the service
- And then the application is ready to receive requests
- Every `@Service`, `@RestController`, and `@Repository` annotation we wrote made this possible
- The annotations register the beans. The constructors declare the wiring. Spring connects everything!

What if Spring finds two beans of the same type?

- If you have two beans of the same type, Spring doesn't know which one to inject and the application will fail to start
- You can resolve this with `@Qualifier`, which lets you specify the bean by name
- You can also solve it with `@Primary`, which gives one bean priority over another in case of ambiguity
- In our conference app and Northwind Traders project, this hasn't been an issue because we have one repository per entity, one service per entity, and one controller per entity

Module 5

Continued Learning

Section 5–1

Authentication

What are JWT tokens?

- **JSON Web Tokens (JWTs) are compact, self-contained tokens used for securely transmitting information between parties.**
 - They are commonly used for authentication and authorization in web applications.
- **JWTs are stateless, scalable, and can be used across different domains.**
- **Think of a JWT like a digital ID card that proves who you are and what you're allowed to do.**
- **A JWT consists of three parts separated by dots (.):**

```
header.payload.signature
```

Example

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJqb2huZG9liiwicm9sZSI6IkFETUI0In0.x5H2vf3QK9_mXxH6YjMo7ZhjKRDtWbK
```

- **Header:** Contains the token type and signing algorithm
- **Payload:** Contains the actual data (username, roles, expiration time)
- **Signature:** Ensures the token hasn't been tampered with

- **How JWTs Work?**

- When a user logs in with username/password, the server validates credentials
- If valid, a JWT is generated containing user info and roles
- This token is sent back to the client
- The client includes this token in the `Authorization` header for all future requests
- The server verifies the token's signature and extracts the user data

- **JWT's in SpringBoot**

- The `jsonwebtoken` library provides classes for creating, parsing, and verifying tokens.

- **Setting up jsonwebtoken Library**

Add to your `pom.xml`:

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt</artifactId>
  <version>0.9.1</version>
</dependency>
```

Protecting Endpoints

- **An example of protecting endpoints:**

```
@RestController
@RequestMapping("/api")

public class UserController {

    @GetMapping("/public/welcome")
    public String publicEndpoint() {
        return "This is a public endpoint - no authentication needed!";
    }

    // Protected endpoint - only users with ADMIN role
    @GetMapping("/admin/users")
    @PreAuthorize("hasRole('ROLE_ADMIN')")
    public String adminOnly() {
        return "Secret admin data - you must be an admin to see this!";
    }

    // Protected endpoint - only authenticated users
    @GetMapping("/user/profile")
    @PreAuthorize("hasRole('ROLE_USER')")
    public String userProfile() {
        return "User profile data - you must be logged in!";
    }
}
```

Section 5-5

Practice Repository

Practice Problems

- We have a repository with additional practice problems and projects
- Your Instructor will inform you which exercises and projects you will work on within this repository
- If the files contain -optional, you are more than welcome to work on these as you please
- <https://github.com/Pluralsight-ILT/YUU-Java-Academy-Bonus>